Unit – 1
Chapter – 1 Introduction and Overview

Data structures are core building blocks of algorithms and real life applications. In simple terms, a well designed algorithm using data structure is like a good structure of a house. For example, to design a good house, we need to put all the components of house together based on the principles of good construction techniques. In the same way, a design of an algorithm must be based on a good understanding of data structures.

The data structure isn't a programming language like C, C++, Java, etc. It is a set of algorithms or techniques or methods that can be used in any programming language to organize the data in the memory.

1.2 Elementary Data Organization

A computer is an electronic machine that processes and manipulates information. The study of computers focuses on how information is organized, manipulated, and used.

In computers, information can be measured. The basic unit of information is the bit, which can be in either the ON or OFF state. The binary digits 0 and 1 are used to represent these states. ON is represented as 1, and OFF is represented as 0. This ON or OFF position is one bit of information. Generally,

```
Information = Instruction + Data
```

? What is Data?

Data is a part of information. It consists of values that are stored either temporarily in a program's data area or permanently on storage devices. Data is a fact or entity with attributes (properties). The attributes are assigned values based on the instructions given. These values can either be characters or numbers.

Example :

- A student's Roll No., Name, Date of Birth, and City are individual data values.
- **Data:** 101 (Roll No.), Srikanth (Name), 05/05/77 (Date of Birth), Bangalore (City).

? What is Elementary Data Organization?

Elementary Data Organization refers to the basic way data is structured and organized in a computer system to make it useful and easy to manage. The organization determines how data is stored, accessed, and processed.

Example: Consider "Student" as an entity. It has attributes like Roll No., Name, Date of Birth (DoB), Area, and City.

	Field 1	Field 2	Field 3	Field 4	Field 5
	Roll No.	Name	DoB	Area	City
Record 1	101	Srikanth	05/05/1977	Girinagar	Bangalore
Record 2	102	Snigdha	07/11/2003	Shankarapuram	Kadapa
Record 3	103	Manjula	07/05/1972	Jayanagar	Bangalore
Record 4	104	Srinath	05/04/1975	Bazar Street	Delhi

Student File

Components of Elementary Data Organization:

1. **Field:**
   - A field is the smallest unit of data that represents a single attribute of an entity.
   - **Example**: In student data, attributes like Roll No., Name, and City are fields.
2. **Record:**
   - A record is a collection of related fields that describe an entity.
   - **Example:** A student's record contains fields like Roll No., Name, Date of Birth, and City.

Roll No.	Name	DoB	City
101	Srikanth	05/05/1977	Bangalore

3. **File:**
   - A file is a collection of records related to the same entity.
   - **Example:** A student file consists of multiple student records.

Roll No.	Name	DoB	City
101	Srikanth	05/05/1977	Bangalore
102	Snigdha	07/11/2003	Kadapa

4. **Primary Key:**
   - A field that uniquely identifies a record in a file is called the primary key.
   - **Example:** "Roll No." is the primary key in the student file because no two students can have the same Roll No.
5. **Entity and Entity Set:**
   - An entity is an object or thing with distinct attributes.
   - A group of entities with similar properties forms an entity set.
   - Example: A "Student" is an entity. A group of students forms a "Student" entity set.

Types of Records:

The records are classified according to their lengths :

(i) **Fixed Length Records**: Fixed-length records are used when each record has a predefined structure and the fields have a fixed size. This ensures uniform space allocation for all records.

Example: Banking System

- In a bank, customer account information often follows a standard format.
- Each record may have fixed fields like Account Number, Name, Account Type, and Balance.
- Regardless of the length of the name or balance, the system allocates the same amount of memory for every record.

Account No.	Name	Account Type	Balance
1012345678	Rajesh Kumar	Savings	50,000
1012345679	Neha	Current	1,00,000

- In this example, each record uses equal space. Even if "Rajesh Kumar" is shorter than "Neha" space allocated for the Name field remains constant.

(ii) Variable Length Records:

Variable-length records are used when data fields can vary in size. This allows flexible memory allocation based on the actual size of the data.

Example: Email Storage System

- In an email system, emails contain Subject, Sender, and Message Body fields.
- The Message Body size varies depending on the content length. A short message takes less space, while a long email requires more space.

Email ID	Name	Subject	Message Body
001	Srikanth	Meeting Update	The meeting is at 4 PM.
002	Swati	Project Proposal	Please find attached the full project report with details...

- In this case, the Message Body for Srikanth's email is short, so less space is used. Swati's email has a longer message, so it uses more space.

1.3 Definition : Data Structure

A data structure is a systematic way to organize, manage, and store data so that it can be used efficiently. It defines how data values like integers, floats, and characters are logically related to each other. Some data values may appear adjacent or close to each other, while others may be positioned far apart but still maintain logical relationships. A data structure not only organizes data but also supports operations like inserting, deleting, searching, and modifying data. It acts as the backbone for efficient data handling in computer systems.

```
Data Structure = Organized Data + Possible Operations on Organized Data
                              (or)
      The Organized Collection of Data is Known as Data Structure.
```

Definitions: Data Structure

1. A data structure is a systematic way to organize, manage, and store data so that it can be used efficiently.
2. A data structure is a way of representing the logical relationships between individual data elements that are used to organize and store data in a computer.
3. A data structure can be defined as the logical (or) mathematical model of a particular organization of data.
4. A data structure is an organization of data and a set of algorithms for performing operations on that data.
5. A data structure is the logical arrangement of data combined with a set of rules or operations for accessing and manipulating that data

1.4 Applications (or) Characteristics (or) Importance of Data Structures

Designing and using data structure is good programming skill, the data structure skills are not provided in general purpose programming languages. The programmer who wants to use these structure has to build them. The data structure is necessary for following reasons:

1. It provides a function that can be used to retrieve the individual data elements.
2. The data structure enables to solve the relationships between the data elements that are relevant to the solution of the problem.
3. Data structure helps to describe the operation that must be performed on the logically related data elements, the operations such as creating, displaying, inserting, deleting, retrieving, etc.
4. Data structure enables to devise the methods of representing the data elements in the memory of the computers, that will reduce the loss of fragmentation and also allows to select the memory configuration or storage structures.
5. Data structures give the freedom to the programmer to decide any type of language that best suits for a particular problem.
6. The algorithm can be improved by structuring the data in such a way that the resulting operations can be efficiently carried out.
7. The data structure describes the physical and logical relationship between the data items. It also provides a mode of access to each element in the data structure.
8. Data structure helps in selection of an appropriate mathematical model for writing a program.



Application Areas

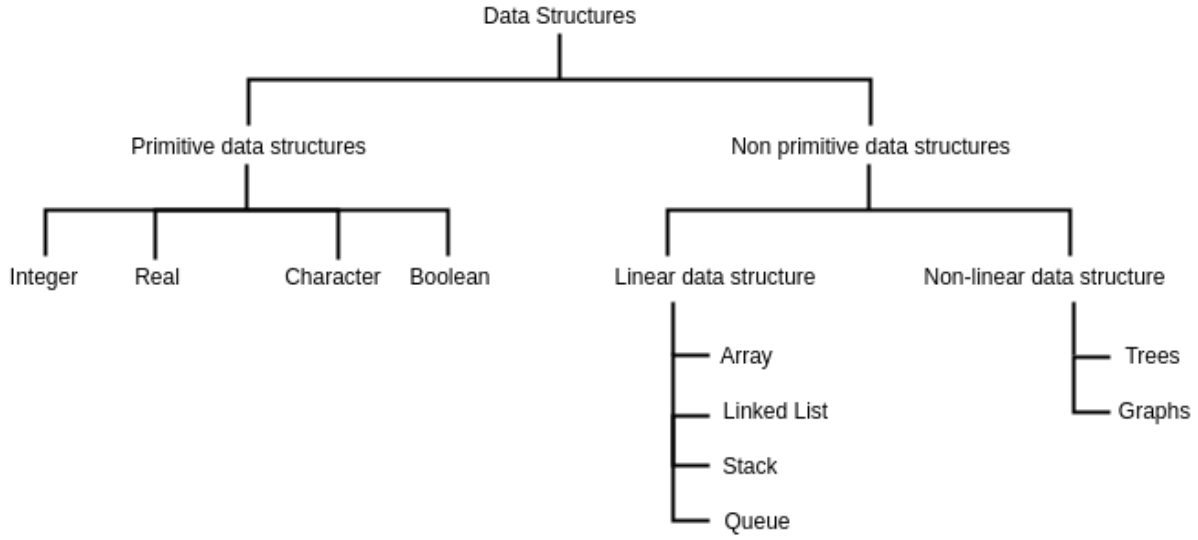
The Data Structures are used in every program or software but they are applied extensively in the following areas:

- Compiler Design
- Artificial Intelligence
- Statistical Analysis Packages
- Operating Systems (OS)
- Game Development
- Embedded Systems
- Database Management Systems (DBMS)
- Network and Numerical Analysis
- Graphics
- Simulations
- Cryptography
- Machine Learning

There are many applications in which different data structures are used for their operations. Some data structures sacrifice speed for efficient utilization of memory, while others sacrifice memory utilization and result in faster speed. In today's world programmers aim not just to build a program, but instead to build an effective program.. As previously discussed, for a program to be efficient, a programmer must choose appropriate data structures.

Classification of Data Structures

Data Structures are generally categorized into two classes: Primitive and Non-Primitive Data structures.



Primitive Data Structure

Primitive data structures are the data structures that can be directly operated on by machine-level instructions. They are the fundamental data types supported by a programming language. The terms *data type*, *basic data type*, and *primitive data type* are often used interchangeably.

Some common primitive data types are **integer**, **real**, **character**, and **boolean**.

In the C programming language, the primitive data structures include **int**, **float**, **char**, and **double**.

For example:

- int age = 10;
- float aggregate = 79.99;
- char gender = 'M';
- double salary = 3458796.00;

Primitive data structures support several basic operations.

- **Creation**: Creation means making a data structure.
  Example: int i;
  This statement creates memory space for the variable i.

- **Deletion (Destruction):** Deletion means removing a data structure to free memory. It helps in saving memory. In C language, the free() function is used to delete dynamically allocated data.
- **Selection**: Selection means accessing or using the data stored in a data structure. Example: reading the value of a variable.
- **Update**: Update means changing the value of data in a data structure.
  Example: int i = 0;
  This statement updates the value of the variable i.

Non-Primitive Data Structures

Non-primitive data structures are data structures that are created using primitive data types. These data structures are used to store and organize a large amount of data efficiently. Examples of non-primitive data structures are arrays, stacks, linked lists, queues, trees, and graphs.

Non-primitive data structures are mainly classified into two types:

1. Linear Data Structures
2. Non-Linear Data Structures

Linear Data Structures

A linear data structure is a data structure in which elements are arranged in a sequential or linear order, one after another. Each element has a unique predecessor and successor, except the first and last elements.

Examples of linear data structures are:

1. Arrays 2. Linked Lists 3. Stacks 4. Queues

Linear data structures can be stored in memory in two ways:

- By using sequential memory locations
- By using links between elements

(i)Arrays

An array is a collection of elements of the same data type stored in contiguous (continuous) memory locations. It is a linear data structure because its elements are stored in a linear order.

Each element in an array is accessed using an index value, also called a subscript.

A one-dimensional array is called a linear array because each element is accessed using one subscript.

A two-dimensional array uses two subscripts to access elements.
Arrays can also be multi-dimensional, such as 2-D, 3-D, 4-D, or N-D arrays.

Example: int A[10] ; //Here, A is the array name and it can store 10 integer elements.

(ii) Linked List

A linked list is a collection of nodes arranged in a sequence. Each node contains data and a pointer that points to the next node in the list. Linked lists are dynamic, which means memory is allocated only when a new node is added.

A linked list stores data elements of the same type in a linear order. There are different types of linked lists such as singly linked list, doubly linked list, and circular linked list.

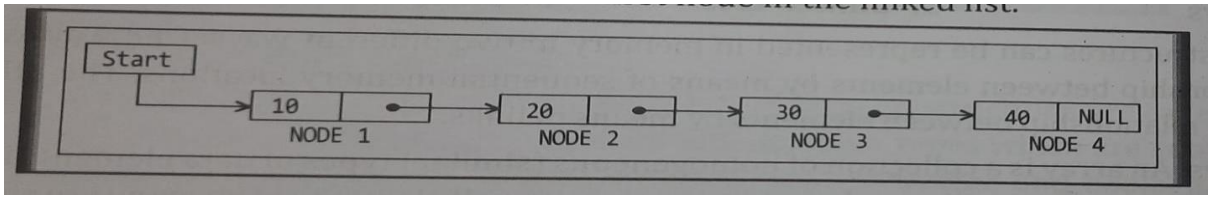
In a linked list, memory for each node is allocated at the time it is added to the list. Every node points to the next node, forming a chain-like structure.

Each node in a linked list has two parts:

1. Data (Information) Field
   It stores the actual value or data of the node.
2. Link (Pointer) Field
   It stores the address of the next node in the list.

A linked list also uses an external pointer called start (or head), which stores the address of the first node in the list. The pointer start points to the first node.

In the linked list, the link field of the last node contains NULL instead of the address of another node. This NULL pointer indicates the end of the linked list.



Real Life Example: In train each coach is connected to its previous and next coach (except first and last coach)

(iii) Stacks

A stack is a linear data structure in which insertion and deletion of elements are performed at only one end, called the top of the stack.

A stack follows the LIFO (Last In, First Out) principle. This means the element that is inserted last is removed first.

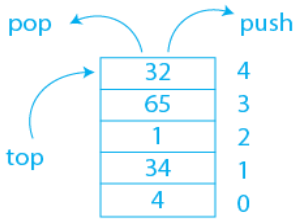
In computer memory, a stack can be implemented using arrays or linked lists.

In a stack, the following three operations are performed:

1. PUSH – Used to insert an element at the top of the stack.
2. POP – Used to remove an element from the top of the stack.
3. PEEP – Used to view the topmost element of the stack without deleting it.

Every stack has a variable called TOP.
The TOP pointer stores the position of the topmost element in the stack.
All insertion and deletion operations take place only at the TOP.

Real-life example:
A stack of plates placed on a table. The plate kept last is removed first.



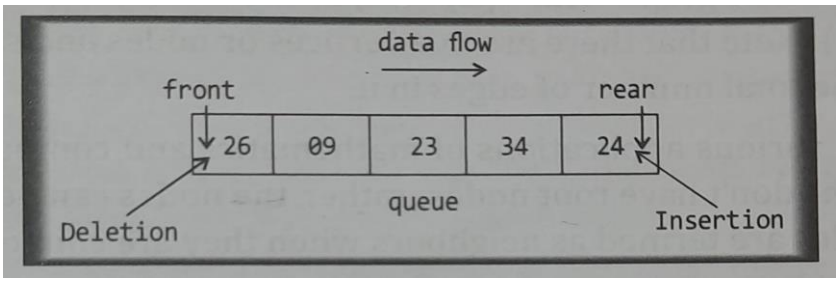
(iv) Queues

A queue is a linear data structure that follows the FIFO (First In, First Out) principle.

This means the element that is inserted first is removed first.

In a queue:

- Insertion of elements takes place at the REAR end (also called the tail).
- Deletion of elements takes place from the FRONT end (also called the head).



Non – Linear Data Structures

A non-linear data structure is a data structure in which elements are not stored in a sequential order. The relationship between elements is not based on adjacency. This means one element is not directly connected only to the next element.

Examples of non-linear data structures are:

1. Trees
2. Graphs

(i)Trees

A tree is a non-linear data structure in which data elements (nodes) are arranged in a hierarchical structure.

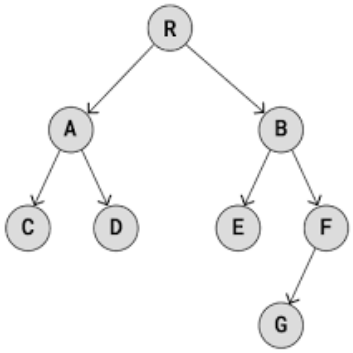
- The topmost node is called the root node.
- The remaining nodes are divided into sub-trees.
- Each sub-tree is also a tree.

Binary Tree

The simplest form of a tree is a binary tree.

- A binary tree consists of:
    - One root node
    - One left sub-tree
    - One right sub-tree
- Each node contains:
    - A data element
    - A left pointer (points to the left sub-tree)
    - A right pointer (points to the right sub-tree)

If root = NULL, then the tree is said to be empty.

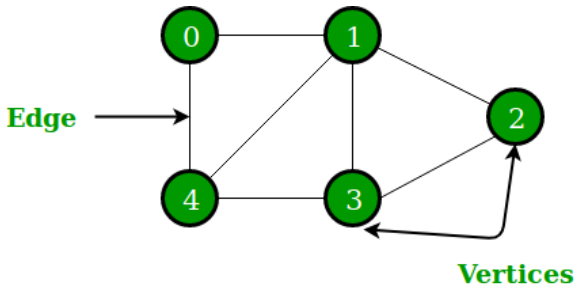


(ii) Graph

A graph is a non-linear data structure that does not follow a parent-child relationship like a tree.

A graph consists of:

- Vertices (nodes)
- Edges that connect the vertices



A graph is represented as: G = (V, E)

where:

- V is the set of vertices
- E is the set of edges
- Graphs can represent complex relationships, and any node can be connected to any other node. Unlike trees, graphs do not have a root node.
- Two nodes are called neighbors if they are connected by an edge.
- The size of a graph is the total number of edges present in it.

Real-life example: A computer network where multiple computers are connected to each other.

Operations on Data Structures

Operations are used to **process and manage data** stored in data structures.

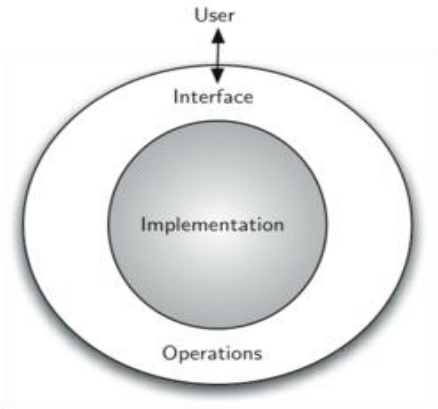
Basic Operations:

1. **Insertion**: Adding a new element to the data structure.
   Example: Adding a new student's details to a class list.
2. **Deletion**: Removing an element from the data structure.
   Example: Deleting the record of a student who left the course.
3. **Traversal**: Visiting each element exactly once to process it.
   Example: Printing all students' names in a class.
4. **Searching**: Finding a specific element in the data structure.
   Example: Finding students who scored 100 marks in C programming.
5. **Sorting**: Arranging elements in ascending or descending order.
   Example: Arranging student names alphabetically or ranking scores.
6. **Merging:** Combining two sorted lists into one single sorted list.

Abstract Data Type (ADT)

- An Abstract Data Type is a type of data where the operations that can be performed are defined, but the details of how the data is stored or implemented are hidden.
  OR
  An Abstract Data Type (ADT) defines the type of data and the operations that can be performed on that data, without specifying how these operations are implemented. It focuses only on what operations are allowed, not on how they are carried out.
- An Abstract Data Type focuses on what operations are performed on the data rather than how they are implemented.
- It provides a logical view of the data and hides the internal details from the user. The implementation of an ADT can vary, such as using arrays or linked lists, but this does not affect how the ADT is used.
- By separating data definition from implementation, ADTs support data abstraction, improve program readability, and make the code easier to maintain and modify.

Working of Abstraction DataType (ADT)



1. User:

The user interacts with the Abstract Data Type through the interface. The user only knows the operations that can be performed on the data, such as adding data, removing data, or retrieving data.

Example, a Gmail user knows that they can send, receive, and compose emails.

2. Interface:

The interface is the layer that connects the user with the Abstract Data Type. It specifies what

operations can be performed on the data but does not reveal how these operations work internally.

Example, in Gmail, options such as “Compose,” “Send,” and “Delete” are part of the interface.

3. Implementation:

The implementation refers to the actual code or logic that makes the operations work. This part is hidden from the user, and the user does not need to know how the operations are implemented.

Example, when sending an email, the user does not see how the email is routed through servers to reach the recipient.

4. Operations:

Operations are the actions or tasks that can be performed on the Abstract Data Type. For example, for a data type such as a list, the operations include adding an item, removing an item, and searching for an item.

Relationship between ADT and Data Structure

An ADT gives the specification of data and operations, while a data structure provides the actual implementation of that specification. Therefore, a data structure is an implementation of an ADT.

Simple Real-Life Analogy

ATM Machine Example

- **ADT (ATM functions)**
  - Withdraw money
  - Deposit money
  - Check balance

You only know **what you can do**, not how it works inside.

- **Data Structure (Implementation)**
  - How the ATM checks balance
  - How it updates the account
  - How it communicates with the bank server

You never see this part.

Example: Queue

Queue ADT defines the operations:

- enqueue – add an element to the queue
- dequeue – remove an element from the queue

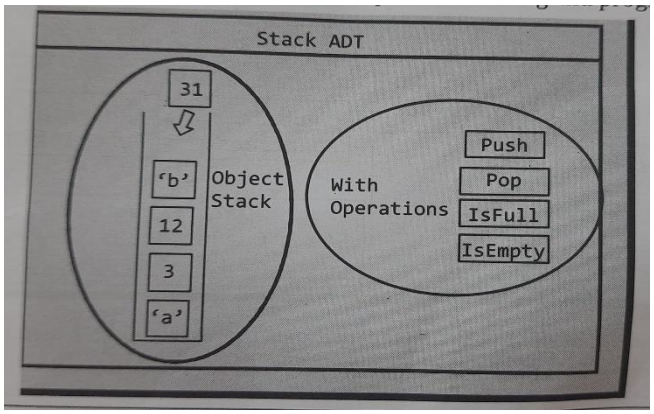
Queue Data Structure implements these operations using:

- Arrays, or
- Linked lists

The operations remain the same, but the implementation method can differ.

Example 1: Stack Abstract Data Type

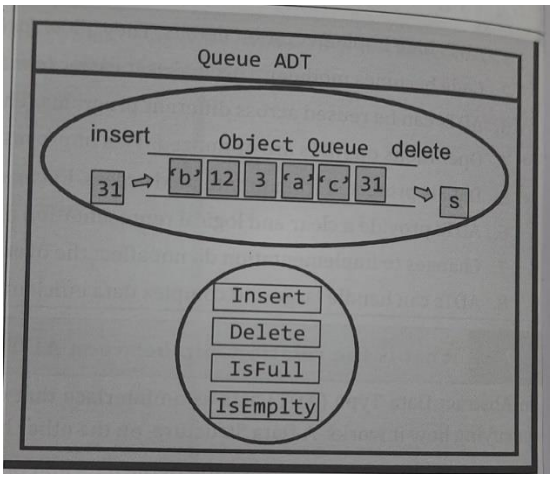
Stack ADT is an Abstract Data Type that stores a collection of elements by following the Last-In, First-Out (LIFO) principle. This means the element that is added last is the first one to be removed. The main operations of a Stack ADT include Push (to add an element to the top of the stack), Pop (to remove the top element), IsFull (to check whether the stack is full), and IsEmpty (to check whether the stack is empty). A stack works like a stack of plates, where the plate placed on top is removed first. The internal implementation of the stack is hidden from the user, and the user interacts with the stack only through these defined operations. Due to its simplicity and efficiency, the Stack ADT is widely used in programming and problem-solving.



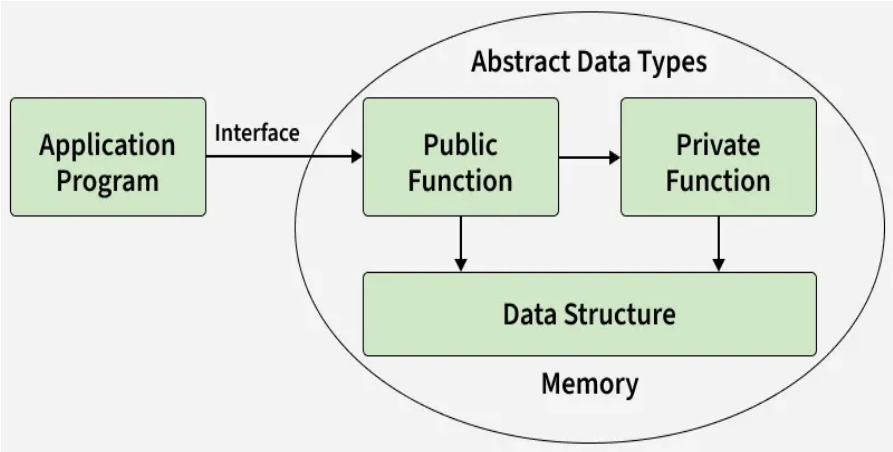
Example 2: Queue Abstract Data Type

A Queue ADT is an Abstract Data Type that organizes a collection of elements using the First-In, First-Out (FIFO) principle. This means the first element inserted into the queue is the first one to be removed. The primary operations supported by a queue are:

- Insert (Enqueue): Adds an element to the rear of the queue.
- Delete (Dequeue): Removes an element from the front of the queue.
- IsFull: Checks if the queue is full and cannot accept more elements.
- IsEmpty: Checks if the queue is empty and has no elements



Abstract Data Type Model



An Abstract Data Type (ADT) model shows how data and operations are organized.

An ADT contains two types of functions:

- Public functions
- Private functions

Private functions can be used only inside the ADT. They cannot be accessed directly by the user.

Public functions are available to the application program or user. The user accesses these functions through an interface.

The interface acts as a bridge between the user and the ADT. It allows the user to call only the public functions.

The functions defined in the ADT are implemented using data structures such as:

- Stack, Queue, Array, Linked List, Trees, Graphs

All data operations like inserting, accessing, modifying, and deleting data are done through the application programming interface (API).
The API can access only public functions, not private functions.

Algorithm

An algorithm is a set of step-by-step instructions to solve a problem. It tells the computer exactly what to do, in the right order, and finishes in a limited amount of time. We need algorithms to solve problems step by step in a clear and easy way.

OR

An Algorithm is a step-by-step procedure for performing some task in a finite amount of time.

Algorithm Complexity

Complexity of an algorithm is a measure of the amount of the time and / or space required by an algorithm for an input of a given size (n).

To understand the complexity of an algorithm, we mainly consider two types of efficiency:

1. **Space Complexity**: Space complexity shows how much memory or storage an algorithm needs while it is running.
   In simple words, it is the amount of memory required by the algorithm to finish its work.
2. **Time Complexity**: Time complexity shows how much time an algorithm takes to complete its execution.

In computer programming, we should always focus on two important things:

(i) Choosing the right data structure, and
(ii) Choosing the right algorithm.
For a single problem, there can be more than one algorithm.
So, we should always choose an efficient algorithm by carefully considering both time complexity and space complexity.

Understanding Space Complexity

Let us assume that Rama and Sita have written two algorithms to perform binary search.

- Rama's algorithm uses 30 bytes of memory to run.
- Sita's algorithm uses 50 bytes of memory to run.

Now, we compare the space used by both algorithms.

Since Rama's algorithm uses less memory than Sita's algorithm,
Rama's algorithm is more space efficient.

Understanding Time Complexity

Assume a program takes 1 second to process one item in an array.

Now, let us see how time complexity affects the total time.

1. O(1) – Constant Time

The program always takes 1 second, no matter how many items are in the list.
Whether the list has 1 item or 100 items, the time remains the same.
This is because the time does not depend on the size of the list.

2. O(n) – Linear Time

The time taken increases as the number of items increases.

- 1 item → 1 second
- 10 items → 10 seconds
- 100 items → 100 seconds

Here, more data means more time.

3. $O(n^2)$ – Quadratic Time

The time taken increases as the square of the number of items.

- 1 item → 1 second
- 10 items → 100 seconds
- 100 items → 10,000 seconds

This type of algorithm becomes very slow for large data.

In real life, programs may have thousands or millions of items.
So, algorithms with higher time complexity take much more time and are inefficient.

Therefore, while designing an algorithm, we must always consider time efficiency and space efficiency to get better performance.

Time Space Trade-Off

The best algorithm is one that uses less memory and takes less time to finish. But in real life, it is not always possible to get both at the same time.

Let us consider two algorithms that solve the same problem:

- The first algorithm uses less memory but takes more time to run.
- The second algorithm runs faster but uses more memory.

This situation is called a time–space trade-off.

- If memory saving is more important, we choose the first algorithm, even though it is slower.
- If speed is more important, we choose the second algorithm, even though it uses more memory.

Definition: Time–space trade-off is a method where we balance time and memory while solving a problem.
An algorithm can:

- run faster by using more memory, or
- use less memory by taking more time.

Example:

- If an algorithm is slow but uses less memory, we can make it faster by using extra memory.
- If memory is limited, we design the algorithm to use less space, even if it becomes slower.

The choice of algorithm depends on the system limits and the application requirement.